

Informe de la Práctica 03

Informe de la Práctica 03: Construcción automática

Curso: Ingeniería de Software II

Práctica: 03 — Construcción automática

Proyecto: SoftwareTextil

Herramienta principal: GNU Make

Rama: doc/automatic

Fecha de elaboración: 20-09-2026

1. Resumen

La práctica implementa un modelo de construcción automática para SoftwareTextil, un sistema web compuesto por un backend Django/DRF, una SPA React construida con Vite y una base de datos PostgreSQL. GNU Make coordina la instalación bloqueada de dependencias, las validaciones, las pruebas, la construcción del frontend, la recolección de estáticos y el empaquetado de la aplicación.

El artefacto definido por el modelo es una distribución ZIP de la aplicación, no un ejecutable nativo ni un paquete wheel. El ZIP contiene el código Django, el frontend compilado, los estáticos recolectados, los metadatos de Python, un manifiesto y un checksum SHA-256.

2. Competencias y objetivo

2.1 Competencia del curso relacionada

La solución aplica herramientas de ingeniería de software para construir, probar, documentar y versionar un proyecto real. También incorpora control de dependencias mediante archivos lock, control de cambios con Git y pruebas automatizadas. No se añadió integración continua porque el PDF la menciona como parte de la competencia general, pero no la exige como entregable de esta práctica.

2.2 Competencia de la práctica

Se construye SoftwareTextil mediante un modelo de construcción ejecutable, declarado principalmente en `Makefile` y complementado por la configuración de `pyproject.toml`.

2.3 Objetivo

Crear y ejecutar un modelo que describa el proyecto, sus herramientas, dependencias, validaciones y empaquetado, de modo que otro integrante pueda reproducir el proceso desde la raíz del repositorio con:

```
make build
```

3. Equipos y materiales

Recurso	Uso en la práctica
Computadora Linux Arch x86_64	Entorno local de ejecución.

Git/GitHub	Rama, commits y control de cambios.
GNU Make 4.4.1	Orquestación del modelo de construcción.
Python 3.11.15 y uv 0.9.10	Backend, dependencias, Ruff, pytest y scripts.
Node.js 26.7.0 y npm 12.0.2	Dependencias y construcción React/Vite.
Docker Compose 5.5.0	Preparación prevista de PostgreSQL 16.
IDE/editor	Edición de configuraciones y documentación.
Internet	Descarga de dependencias y consulta de documentación oficial.

El daemon de Docker no estuvo disponible durante la ejecución final; por eso la verificación de PostgreSQL se reporta como pendiente y no como realizada.

4. Prerrequisitos

1. Clonar el repositorio y cambiar a `doc/automatic`.
2. Tener instalados Git, Make, uv, Python 3.11, Node/npm y Docker Compose.
3. Copiar `.env.example` como `.env`, sin versionar secretos.
4. Iniciar PostgreSQL 16 con `docker compose up -d --wait db`.
5. Contar con un usuario PostgreSQL que pueda crear la base temporal de pytest-django.

La instalación detallada y los comandos reproducibles están en [tutorial.md](#).

5. Herramienta de construcción seleccionada

Se seleccionó GNU Make porque el proyecto combina dos ecosistemas y necesita coordinar comandos heterogéneos: uv para Python y npm/Vite para React. Make permite expresar dependencias entre etapas sin sustituir las herramientas especializadas de cada lenguaje.

Maven y CMake, presentados como ejemplos en el PDF, no son apropiados como herramienta principal aquí: Maven está orientado a Java y CMake a proyectos que usan C/C++. Incorporarlos habría duplicado la gestión existente y no habría descrito fielmente el proyecto Django/React.

6. Modelo de construcción

6.1 Información del proyecto

La información se obtiene de `pyproject.toml`:

```
[project]
name = "software-textil"
version = "0.1.0"

[tool.uv]
package = false
```

```
[tool.software-textil.release]
format = "zip"
output-dir = "dist"
```

El nombre y la versión se leen directamente desde [project]; no se duplican como constantes en los scripts de empaquetado.

6.2 Opciones de construcción

- `.python-version` selecciona Python 3.11.
- `.node-version` documenta Node 22.14.0 como versión de referencia.
- `frontend/inventario/package.json` declara el motor `^20.19.0 || >=22.12.0`, compatible con Vite 7.
- `frontend/inventario/vite.config.js` conserva la base `/static/zuren/` y los nombres `assets/app.js` y `assets/app.css`.
- `config/settings/build.py` define `STATIC_ROOT` en `build/staticfiles` sin modificar `config/settings/prod.py`.
- `.NOTPARALLEL`, `Bash -eu -o pipefail` y las dependencias de los objetivos evitan carreras y detienen el proceso ante errores.

6.3 Gestión de dependencias

Área	Configuración	Instalación
Python	<code>pyproject.toml</code> y <code>uv.lock</code>	<code>uv sync --locked</code>
Python de desarrollo	grupo dev	<code>pytest</code> , <code>pytest-cov</code> , <code>pytest-django</code> y <code>Ruff</code>
Frontend	<code>package.json</code> y <code>package-lock.json</code>	<code>npm ci</code>
Base de datos	<code>docker-compose.yml</code>	Imagen PostgreSQL 16-alpine

6.4 Flujo de objetivos

`prerequisites`

```
↓
deps → frontend-check → frontend → check → db-ready → test
                                         ↓
                                         static → package
                                         ↓
                                         verify-release → build
```

Objetivos principales: `help`, `deps`, `frontend`, `check`, `test`, `static`, `package`, `verify-release`, `build` y `clean`. `clean` solo elimina salidas generadas y no toca bases, volúmenes ni cambios de Git.

7. Actividades realizadas

Actividad 1: Configuración de la herramienta

Se configuró GNU Make mediante `Makefile` y se comprobó localmente con `make prerequisites`. La salida registró Make 4.4.1, uv 0.9.10, Python 3.11.15, Node 26.7.0 y npm 12.0.2.

Actividad 2: Creación del modelo

El modelo se implementó en:

- `Makefile`: etapas, dependencias, comandos y errores.
- `pyproject.toml`: metadatos y contrato del ZIP.
- `config/settings/build.py`: settings aislados y `STATIC_ROOT`.
- `scripts/package_release.py`: lista explícita, manifiesto y checksum.
- `scripts/verify_release.py`: validación y extracción fuera del checkout.
- `package.json`, `package-lock.json`, `.node-version` y `.python-version`: compatibilidad y reproducibilidad.

Actividad 3: Construcción

Se ejecutaron individualmente las pruebas frontend, ESLint, Vite, checks Django, Ruff focalizado, pruebas del empaquetador, `collectstatic`, empaquetado y verificación del ZIP. La construcción completa `make clean && make build` llegó hasta Ruff, pero se detuvo por 58 errores preexistentes del código general del repositorio. Make actuó conforme al requisito de fallar rápido y no anunció éxito.

Actividad 4: Visualización de resultados

El artefacto fue inspeccionado mediante `unzip`, `sha256sum` y un script Python. Se comprobó:

- ZIP: `dist/software-textil-0.1.0.zip`.
- Checksum final registrado en `dist/software-textil-0.1.0.zip.sha256`.
- 380 entradas en el ZIP y 379 archivos listados en el manifiesto.
- `frontend/inventario/dist/index.html`, `app.js` y `app.css` presentes.
- `build/staticfiles/zuren/assets/app.js` y `app.css` presentes.
- Ausencia de `.env`, `.git`, `.venv`, `node_modules`, `media/` y `db.sqlite3`.
- Extracción en `/tmp/software-textil-release-*` y `manage.py check` con un entorno Python independiente.

8. Pruebas y resultados

Validación	Resultado
Pruebas frontend	4 aprobadas.
ESLint frontend	Aprobado.
Build Vite	Aprobado; produjo JS, CSS e HTML.
Pruebas del empaquetador	12 aprobadas.
Ruff de archivos de la práctica	Aprobado.
Django check con settings de build	Sin incidencias.
Revisión de migraciones	Sin cambios detectados; base no disponible.
ZIP, manifiesto y checksum	Verificados.

Extracción y entorno independiente Verificados.

PostgreSQL y suite Python completa Pendientes por daemon Docker inactivo y bloqueo previo de Ruff.

La evidencia detallada se encuentra en [evidencias.md](#). No se presentan como aprobadas las verificaciones que no pudieron ejecutarse.

9. Entregables solicitados por el PDF

9.1 Proyecto GitHub

- Rama: `doc/automatic`.
- Commit base: `63e48ac75026d682c231efb724f436e265489653`.
- Commits de implementación documentados en [README.md](#).
- Estado: commits locales; publicación pendiente de autorización.

9.2 Modelo de construcción de software

El archivo principal es [Makefile](#). Sus configuraciones asociadas se enlazan en la tabla del [README.md](#), y los scripts auxiliares implementan únicamente las etapas que Make invoca.

9.3 Tutorial

El tutorial en español es [tutorial.md](#). Incluye instalación, configuración, rama, variables de entorno, PostgreSQL, modelo, construcción, artefactos, extracción, limpieza, problemas y documentación oficial.

10. Limitaciones y trabajo posterior

1. Iniciar Docker y PostgreSQL, ejecutar migraciones de una base de práctica y repetir la suite Python completa.
2. Corregir los 58 errores Ruff preexistentes fuera del alcance específico de esta práctica y repetir `make clean && make build`.
3. Publicar la rama con `git push -u origin doc/automatic` cuando el responsable lo autorice.

Estas limitaciones son explícitas y no invalidan la existencia del modelo ni la verificación independiente del empaquetado; simplemente impiden afirmar que el flujo integral actual terminó en estado verde.

11. Conclusiones

GNU Make permitió integrar en un único modelo las herramientas reales del proyecto sin migrar Django, React o PostgreSQL a otra tecnología. El proceso instala dependencias bloqueadas, valida el código, construye la SPA, recopila estáticos, genera un ZIP con metadatos y comprueba su extracción fuera del árbol fuente.

La solución cumple las actividades y entregables documentales del PDF con resultados verificables. La evidencia mantiene una distinción importante entre las etapas que pasaron y los bloqueos externos o preexistentes que deben resolverse para obtener un `make build` completamente verde.

12. Referencias

- [Enunciado de la Práctica 03](#)
- [GNU Make Manual](#)
- [Documentación de uv](#)
- [Documentación de npm ci](#)
- [Guía de build de Vite](#)
- [Archivos estáticos en Django](#)
- [Docker Compose](#)